

Assignment 1

Types

1. Consider the Curry-style simply-typed λ -calculus with Booleans from Lecture 7, extended as follows with natural numbers and a **rec** $f.x.M$ construct.

$$\begin{aligned} M &::= x \mid \lambda x.M \mid M M \mid \text{true} \mid \text{false} \mid \text{if } M \text{ then } M \text{ else } M \\ &\quad \mid \text{zero} \mid \text{succ } M \mid \text{pred } M \mid \text{iszero } M \mid \text{rec } f.x.M \\ T &::= \mathbb{B} \mid \mathbb{N} \mid T \rightarrow T \end{aligned}$$

The typing rules and semantics for $\mathbb{N}$, **zero**, **succ**, **pred**, **iszero** are those from Lecture 9. Intuitively, the term **rec** $f.x.M$ defines a function much like a λ -abstraction, but it binds both variables f and x in M so that f can be used to call the same function recursively inside M . Thus, the semantics of **rec** $f.x.M$ is essentially identical to that of $\lambda x.M$ in the case where f does not appear free in M . The new typing rule is

$$\frac{\Gamma, f : T \rightarrow U, x : T \vdash M : U}{\Gamma \vdash \text{rec } f.x.M : T \rightarrow U} \text{REC}$$

which differs from **ABS** only in the presence of $f : T \rightarrow U$ in the typing environment.

Where these questions ask you to modify definitions, you need only explain the changes that involve the new term **rec** $f.x.M$.

- (a) Check that $\vdash \text{rec } f.x.f(\text{succ } x) : \mathbb{N} \rightarrow \mathbb{B}$ is a valid typing judgement.
 - (b) Extend the definition of the set $\text{FV}(M)$ of free variables in the term M , and the definition of capture-avoiding substitution.
 - (c) Extend the call-by-value small-step operational semantics of the Curry-style simply-typed λ -calculus to account for the new form **rec** $f.x.M$. Use evaluation contexts.
 - (d) Prove the following by writing out the sequence of small steps.
- $(\text{rec } f.x.\text{if iszero } x \text{ then true else if } f(\text{pred } x) \text{ then false else true})(\text{succ zero}) \rightarrow_v^* \text{false}$

[8 marks]

2. Perform type inference using both Wand's and Robinson's algorithms on the following expression and conclude whether it is typable within the simply-typed λ -calculus. Explain carefully (either by labelling an incomplete derivation tree or working with a set of goals) how each constraint arises from Wand's algorithm. In each step of Robinson's algorithm indicate which type equality you have chosen to analyse.

$$\lambda x.\text{if } x \lambda y.\text{true then } x \text{ else } x$$

[6 marks]

3. In System F, we can encode 'Maybe' types with

$$\text{Maybe}(\alpha) \equiv \forall \beta.(\alpha \rightarrow \beta) \rightarrow \beta \rightarrow \beta$$

Write down a term **Just** and a typing derivation of $\vdash \text{Just} : \forall \alpha. \alpha \rightarrow \text{Maybe}(\alpha)$. Your term should be one that satisfies $(\text{Just}\{\mathbb{N}\}(\text{succ zero}))\{\mathbb{N}\}(\lambda x : \mathbb{N}.x) \text{zero} \rightarrow_v^* \text{succ zero}$, but you do not need to give a proof of this.

[6 marks]